

```
!pip install -q transformers torch scikit-learn
```

```
import torch
import numpy as np
from torch.utils.data import Dataset, DataLoader
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, f1_score
from transformers import DistilBertTokenizerFast, DistilBertForSequenceClassification
from torch.optim import AdamW
```

```
texts = [
    "I love this movie",
    "This film was amazing",
    "Absolutely fantastic experience",
    "I really liked the story",
    "Best movie ever",
    "I hate this movie",
    "Worst film I have seen",
    "Terrible acting",
    "Not worth watching",
    "Very disappointing"
]

# 1 = Positive, 0 = Negative
labels = [1,1,1,1,1, 0,0,0,0,0]
```

```
train_texts, test_texts, train_labels, test_labels = train_test_split(
    texts, labels, test_size=0.3, random_state=42
)
```

```
tokenizer = DistilBertTokenizerFast.from_pretrained("distilbert-base-uncased")
```

```
/usr/local/lib/python3.12/dist-packages/huggingface_hub/utils/_auth.py:94: UserWarning:
The secret `HF_TOKEN` does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your settings tab (https://huggingface.co/settings/tokens)
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to access public models or datasets.
warnings.warn(

tokenizer_config.json: 100% 48.0/48.0 [00:00<00:00, 3.33kB/s]

Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable f
WARNING:huggingface_hub.utils._http:Warning: You are sending unauthenticated requests to the HF Hu
vocab.txt: 232k/? [00:00<00:00, 6.07MB/s]

tokenizer.json: 466k/? [00:00<00:00, 3.87MB/s]
```

```
train_encodings = tokenizer(train_texts, truncation=True, padding=True)
test_encodings = tokenizer(test_texts, truncation=True, padding=True)
```

```
class SentimentDataset(Dataset):
    def __init__(self, encodings, labels):
        self.encodings = encodings
        self.labels = labels

    def __getitem__(self, idx):
        item = {key: torch.tensor(val[idx]) for key, val in self.encodings.items()}
        item["labels"] = torch.tensor(self.labels[idx])
        return item

    def __len__(self):
```

```
        return len(self.labels)

train_dataset = SentimentDataset(train_encodings, train_labels)
test_dataset = SentimentDataset(test_encodings, test_labels)
```

```
train_loader = DataLoader(train_dataset, batch_size=2, shuffle=True)
test_loader = DataLoader(test_dataset, batch_size=2)
```

```
model = DistilBertForSequenceClassification.from_pretrained(
    "distilbert-base-uncased",
    num_labels=2
)

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
model.to(device)
```

config.json: 100%

483/483 [00:00<00:00, 41.4kB/s]

model.safetensors: 100%

268M/268M [00:03<00:00, 154MB/s]

Loading weights: 100% 100/100 [00:00<00:00, 444.90it/s, Materializing param=distilbert.transformer.layer.5.sa_layer_norm.w

DistilBertForSequenceClassification LOAD REPORT from: distilbert-base-uncased

Key	Status
-----+-----+	
vocab_layer_norm.weight	UNEXPECTED
vocab_layer_norm.bias	UNEXPECTED
vocab_transform.weight	UNEXPECTED
vocab_projector.bias	UNEXPECTED
vocab_transform.bias	UNEXPECTED
pre_classifier.weight	MISSING
pre_classifier.bias	MISSING
classifier.weight	MISSING
classifier.bias	MISSING

Notes:

- UNEXPECTED :can be ignored when loading from different task/architecture; not ok if you expected
- MISSING :those params were newly initialized because missing from the checkpoint. Consider tra

DistilBertForSequenceClassification

```
(distilbert): DistilBertModel(
  (embeddings): Embeddings(
    (word_embeddings): Embedding(30522, 768, padding_idx=0)
    (position_embeddings): Embedding(512, 768)
    (LayerNorm): LayerNorm((768,), eps=1e-12, elementwise_affine=True)
    (dropout): Dropout(p=0.1, inplace=False)
  )
  (transformer): Transformer(
    (layer): ModuleList(
      (0-5): 6 x TransformerBlock(
        (attention): DistilBertSelfAttention(
          (q_lin): Linear(in_features=768, out_features=768, bias=True)
          (k_lin): Linear(in_features=768, out_features=768, bias=True)
          (v_lin): Linear(in_features=768, out_features=768, bias=True)
          (out_lin): Linear(in_features=768, out_features=768, bias=True)
          (dropout): Dropout(p=0.1, inplace=False)
        )
        (sa_layer_norm): LayerNorm((768,), eps=1e-12, elementwise_affine=True)
        (ffn): FFN(
          (dropout): Dropout(p=0.1, inplace=False)
          (lin1): Linear(in_features=768, out_features=3072, bias=True)
          (lin2): Linear(in_features=3072, out_features=768, bias=True)
          (activation): GELUActivation()
        )
        (output_layer_norm): LayerNorm((768,), eps=1e-12, elementwise_affine=True)
      )
    )
  )
  (pre_classifier): Linear(in_features=768, out_features=768, bias=True)
  (classifier): Linear(in_features=768, out_features=2, bias=True)
  (dropout): Dropout(p=0.2, inplace=False)
)
```

```
optimizer = AdamW(model.parameters(), lr=5e-5)
```

```
model.train()
```

```
for epoch in range(3):
```

```
    total_loss = 0
```

```
    for batch in train_loader:
```

```
        optimizer.zero_grad()
```

```
        input_ids = batch["input_ids"].to(device)
```

```
        attention_mask = batch["attention_mask"].to(device)
```

```
labels = batch["labels"].to(device)

outputs = model(
    input_ids=input_ids,
    attention_mask=attention_mask,
    labels=labels
)

loss = outputs.loss
total_loss += loss.item()

loss.backward()
optimizer.step()

avg_loss = total_loss / len(train_loader)
```

Epoch 1 Loss: 0.6897
Epoch 2 Loss: 0.5521
Epoch 3 Loss: 0.4306

```
model.eval()
predictions = []
true_labels = []

with torch.no_grad():
    for batch in test_loader:
        input_ids = batch["input_ids"].to(device)
        attention_mask = batch["attention_mask"].to(device)
        labels = batch["labels"].to(device)

        outputs = model(input_ids=input_ids, attention_mask=attention_mask)
```